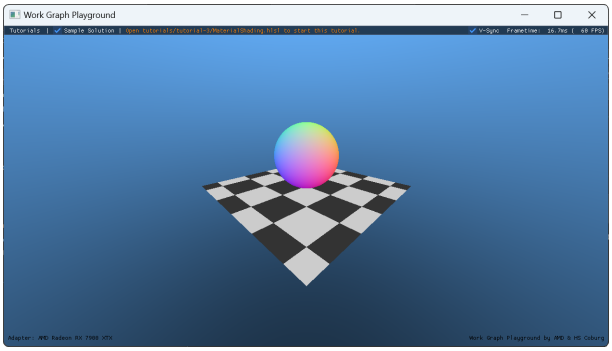
GPU Work Graphs

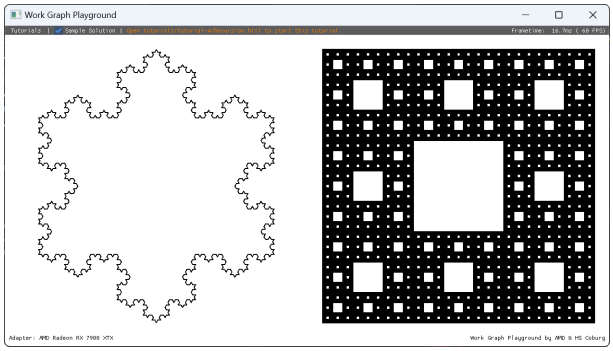
Bastian Kuth*
Coburg University of Applied Sciences and Arts
Coburg, Germany
bastian.kuth@hs-coburg.de

Max Oberberger*
Advanced Micro Devices (AMD)
Garching, Germany
max.oberberger@amd.com

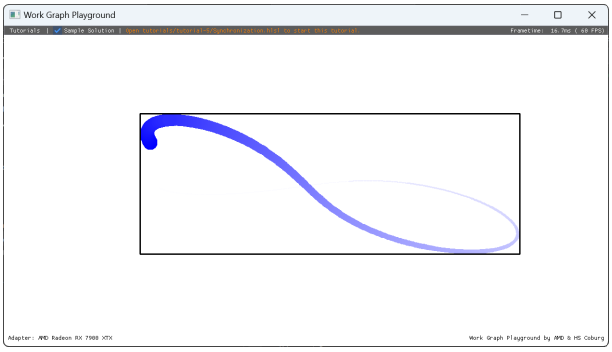
Quirin Meyer†
Coburg University of Applied Sciences and Arts
Coburg, Germany
quirin.meyer@hs-coburg.de



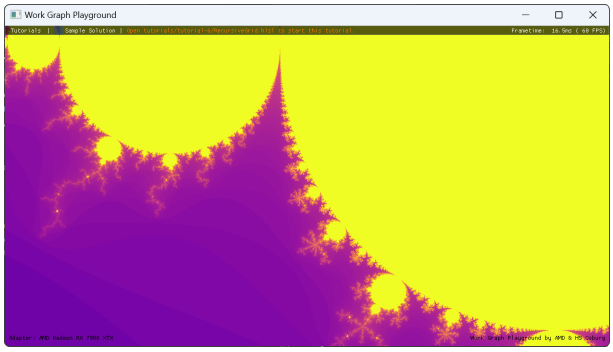
(a) Material shading tutorial. With Work Graphs, the GPU can dispatch different shaders depending on the material.



(b) Recursion tutorial. Work Graphs allow GPU to dispatch work to themselves allowing for trivial recursion.



(c) Synchronization tutorial. Different thread-groups of a broadcasting node communicate amongst each other.



(d) Homework project. Work Graphs allow to compute the Mandelbrot set recursively on the GPU.

Figure 1: Example images from our Work Graph Playground App. We use our Work Graph Playground App to provide easy access to the world of Work Graphs. Participants can focus on Work Graphs shader code only without worrying about boilerplate setup-code. With our app, participants can use their Windows laptops to actively dive into the most important Work Graphs concepts, even without a GPU supporting Work Graphs. We provide hands-on tutorials with tasks, solutions, and detailed descriptions that are suitable for both in-class as well as homework assignments.

Abstract

Work Graphs is a Direct3D12 feature added in 2024 and enables GPU compute shaders to dispatch other shaders directly without CPU intervention. Thus, they can help improve GPU memory usage and runtime speed. In this course, we teach Work Graphs concepts at practical and relevant hands-on examples with a focus on the non-trivial HLSL Work Graphs additions. After this class, participants should be able to assess when Work Graphs is useful for their tasks and be able to understand, explain, and apply Work Graphs.

*Both authors contributed equally to this research.
†Corresponding author.

SIGGRAPH Courses '25, Vancouver, BC, Canada

ACM ISBN 979-8-4007-1543-3/25/08
https://doi.org/10.1145/3721241.3734003

ACM Reference Format:
Bastian Kuth, Max Oberberger, and Quirin Meyer. 2025. GPU Work Graphs. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Courses (SIGGRAPH Courses '25), August 10–14, 2025, Vancouver, BC, Canada.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3721241.3734003

1 Before the Course

Prerequisites to our course are

- the knowledge of HLSL or similar shading language,
- basic knowledge of a modern graphics APIs such as Direct3D12, and
- for the hands-on part, a laptop with Windows 10 or 11 and a code editor.

During the course, we interweave little hands-on samples. While not required, we encourage participants to take part in it for a better learning outcome and experience.

For this, we prepared Work Graph Playground App. Full source-code and documentation is available on GitHub[1]. For quick installation, we provide binaries for download[2]. We recommend installing and testing Work Graph Playground App upfront on your laptop.

2 Introduction

Work Graphs constitute one of the most recently added Direct3D12 features [Microsoft Cooperation 2024]. It was introduced at GDC 2024 [Hargreaves and Chajdas 2024]. The innovation behind Work Graphs is that they enable GPU compute shaders to dispatch other GPU compute shaders directly from the GPU without going back to the CPU. This makes Work Graphs useful for general-purpose computing as well as graphics applications. They are helpful to easily map dependent, recursive, and irregular work-loads on the GPU.

Unlike previous approaches, Work Graphs do this in a fast and memory-efficient way. Work Graphs reduce CPU-GPU data-transfer and synchronization. Further, they allow more algorithms to be expressed in a GPU-only fashion. Ultimately, this speeds up computation and saves memory. In computer graphics, a plethora of applications such as geometry generation, procedural modelling, shading, or decompression, etc. benefit from Work Graphs in terms of implementation simplicity, memory efficiency, and performance.

The biggest change in Work Graphs concerns shader-language additions that demand a thorough explanation. Therefore, in this course, graphic programmers gain an understanding of the necessary fundamentals, concepts, tools, programming techniques, as well as, related concepts to understand, describe, and use Work Graphs. We focus on the HLSL-aspects of Work Graphs. We cover topics like node-records, node-launches, the classify-and-execute pattern, recursions, and synchronization. Furthermore, we provide insight in how a hardware-implementation of Work Graphs could work. Further, we show advanced examples demonstrating the power of Work Graphs.

As an activity, participants experience multiple hands-on programming examples using our easy-to-learn Work Graph Playground App. It is open-source and includes tutorials with detailed descriptions, tasks, and solutions.

After this class, participants should understand, explain, and apply Work Graphs. As main takeaway from this course, attendees should be excited and ready to solve own programming projects with Work Graphs and decide when Work Graphs are useful for their problems at hand.

[1] https://github.com/GPUOpen-LibrariesAndSDKs/WorkGraphPlayground
[2] https://github.com/GPUOpen-LibrariesAndSDKs/WorkGraphPlayground/releases

3 Presenters

The course is designed and presented by three instructors from industry and academia:

Bastian Kuth is wrapping up his PhD at Coburg University of Applied Sciences and Arts and University of Erlangen-Nuremberg. His research focuses on real-time geometry processing on GPU, especially in combination with Work Graphs. He has published in the areas of image-based rendering, virtual reality, geometry compression, subdivision surfaces, and procedural generation.

Max Oberberger is a senior software engineer at AMD's GPU Architecture and Software Technologies Team. His work focuses on research into applications of new GPU technologies, such as Mesh Shaders and Work Graphs.

Quirin Meyer is professor for computer graphics at Coburg University of Applied Sciences and Arts. He obtained his PhD in Computer Graphics from the University of Erlangen-Nuremberg in 2012. His research interests include real-time computer graphics, geometry processing, and parallel computing. Prior to his return to academia in 2017, he applied his interests in industry. In Coburg, he designed the Visual Computing B.Sc. program and serves as its academic director.

4 Course Format

4.1 Part One: Background

- Why Work Graphs?
- A summary of GPU programming concepts
- Teasing videos and real-time demos using Work Graphs
- What problems do Work Graphs solve?
- Comparing Work Graphs against prior solutions, such as Execute Indirect

4.2 Part Two: Work Graphs Concepts Hands-On

(1) **Work Graphs Concepts: Nodes**
   - Introduction to the Work Graph Playground App
   - Detail explanation of nodes
   - Hands-on sample for nodes `HelloWorkGraphs.hlsl`
   - Simple task for participants to do with their laptops

(2) **Work Graphs Concepts: Records**
   - Detailed explanation of Records.
   - Hands-on sample for Records: `Records.hlsl`
   - Instructor solves one sub-problem
   - Instructor explains a task to the participants
   - Attendees solve small problem
   - Explanation of home-work assignment

(3) **Work Graphs Concepts: Launches**
   - Detailed explanation of launches
   - Hands-on example for launches: `NodeLaunches.hlsl`
   - Instructor solves one sub-problem
   - Instructor explains a task to the participants
   - Attendees solve small problem
   - Explanation of home-work assignment

(4) **Work Graphs Use-case: Material Shading**
   - Slides introduce the "Classify-and-Execute" pattern.
   - Introduce example: `MaterialShading.hlsl`
   - Explanation of home-work assignment

(a) (b) (c) (d)

Figure 2: In the third part of the course, we show more advanced Work Graphs use-cases. Image (a) shows a scene without procedural generation. Image (b) shows the same scene is enhanced with our procedural generation using Work Graphs. Images (c) and (d) show examples of procedural foliage generated and rendered using Work Graphs and Mesh Nodes.

4.3 Part Three: Advanced Work-Graphs

(1) **Outlook: Recursion and Synchronization**
- Explain concepts of recursion and synchronization
- Introduction of home-work projects: Recursion and Synchronization

(2) **Application: Procedural Generation with Work Graphs**
- Showcasing the features of Work Graphs with challenging practical examples to generate geometry in real-time.

(3) **Work Graphs under the Hood**
- Potential GPU implementation of Work Graphs
- Work Graphs best practices

(4) **Wrap-Up and Home-Work Projects**
- Introduction of homework projects: Mandelbrot and Mesh Node
- Summary and Questions

5 Description of Course

Our course is based on slides with notes, a cheat sheet, and a tutorial application. Our course subdivides into three parts: a background part, a concepts-hands-on part, and an advanced part. In the first part, we provide fundamentals of GPU programming and architecture required for Work Graphs. We further point out the necessity of Work Graphs and compare it with alternative approaches.

After these basics have been established, the second part starts with the fundamental Work Graphs concepts, namely Graph, Nodes, Record, and Launches. To better understand these concepts, we provide hands-on examples that attendees are encouraged to do in class. We make Work Graphs more accessible by providing a tutorial framework called Work Graph Playground App, shown in Figure 1 and Figure 3. We open-sourced the app on GitHub[3].

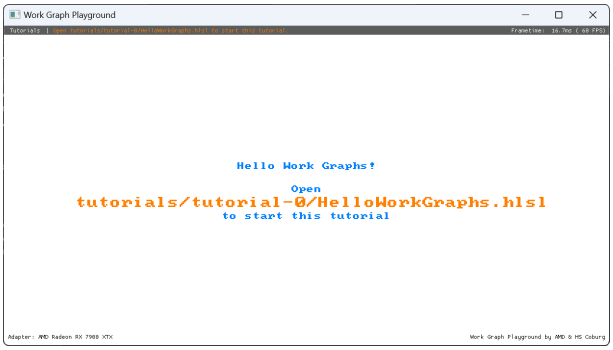
With our tutorials, participants get to know and experience the different aspects of the Work Graphs API. Each tutorial comes with detailed instructions and a sample solution. The tutorials only focus on the GPU side of Work Graphs. Therefore, course members do not have to worry about any of the necessary CPU-side setup. All assignments are solvable with laptops directly during the course, even without a GPU with Work Graphs support. Please remember to bring your laptop to the course and we highly recommend to install the Work Graph Playground App beforehand.

In the third part, we explain and show-case more advanced systems and algorithms that we have created during our research [Kuth et al. 2024, 2025] in work graphs, see Figure 2. Further, we show best practices and provide a view on a potential hardware implementation. We close the course with a question and answer section.

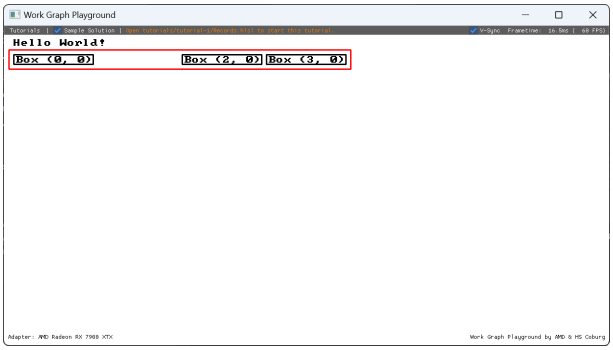
6 Learning Objectives and Outcomes

Upon completion, course attendees will have an understanding of Work Graphs to complete the homework assignments. Further, they

[3] https://github.com/GPUOpen-LibrariesAndSDKs/WorkGraphPlayground



(a) Hello Work Graphs. A basic application introduces Work Graphs and explains the concepts of Nodes.



(b) Tutorial for Records. Records are another core concept of Work Graphs for passing data between Nodes.

Figure 3: More example images from our Work Graph Playground App. Two introductory examples that are part of our practical hands-on samples.

should be able to start their own projects as well as explain Work Graphs to peers. This course should further empower practitioners to transfer and apply the concepts to other APIs, such as Vulkan, which expose similar features [Vulkan Working Group 2024]. Finally, participants should be able to identify applications for Work Graphs in their own research or work contexts.

References

Shawn Hargreaves and Matthäus G. Chajdas. 2024. GPU Work Graphs: Welcome to the Future of GPU Programming. In *Game Developer Conference 2024*. San Francisco, CA, USA. https://gpuopen.com/gdc-presentations/2024/GDC2024_GPU_Work_Graphs.pdf

Bastian Kuth, Max Oberberger, Carsten Faber, Dominik Baumeister, Matthäus Chajdas, and Quirin Meyer. 2024. Real-Time Procedural Generation with GPU Work Graphs. *Proc. ACM Comput. Graph. Interact. Tech.* 7, 3 (Aug. 2024).

Bastian Kuth, Max Oberberger, Carsten Faber, Pirmin Pfeifer, Seyedmasih Tabaei, Dominik Baumeister, and Quirin Meyer. 2025. Real-Time GPU Tree Generation. In *Proceedings of High-Performance Graphics (HPG)*. ACM, Copenhagen, Denmark.

Microsoft Cooperation 2024. *D3D12 Work Graphs*. Microsoft Cooperation. https://microsoft.github.io/DirectX-Specs/d3d/WorkGraphs.html

Vulkan Working Group. 2024. VK_AMDX_shader_enqueue: Shader Enqueue Proposal. https://docs.vulkan.org/features/latest/features/proposals/VK_AMDX_shader_enqueue.html.